

```
$ curl /users/123 | usecase | repository | db
```

Clean Architecture

// 0000000000

Controller

□□

Usecase

□□

Repository

□□

Presenter

□□□

\$ cat blueprint.md



01 /



02 /



03 /



TypeScript



04 /



DB



PART 01



□□□□□□□□



```
1 // UI 로직
2 app.get('/users', (req, res) => {
3   const users = db.query('SELECT * FROM users');
4   res.json(users.map(u => ({
5     name: u.name,
6     email: u.email,
7     createdAt: u.created_at.format('YYYY-MM-DD')
8   })));
9 });
```

INCOMING REQUESTS

GraphQL

→

→ 10

MongoDB

→ SQL

NOGLYPH NOGLYPH



10

NO GMPH

10

NO 514PH NO 514PH



Usecase — 使用例

Usecase — 使用例

PART 02





Controller = □□□



Usecase = □□



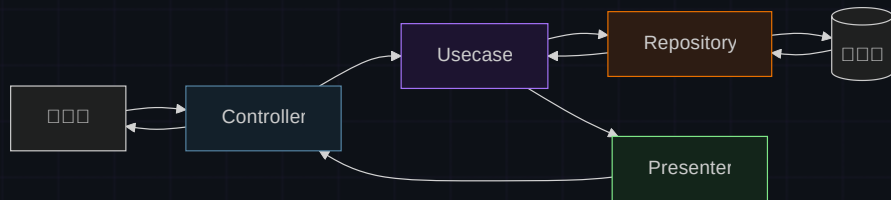
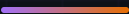
Repository = □□



Presenter = □□□



□ □ □ □ □ □ □ □ □ □



□ □

UI → Usecase → Repository → DB

□ □

DB → Repository → Usecase → Presenter → UI □ □ □ □

PART 03

TypeScript □□

□□□□□□□□□□

Repository

NO SHIP

□□□□

```
1 export interface UserRepository {
2   getUser(id: number): Promise<User | null>;
3 }
4
5 // MySQL — Usecase
6 export class MySQLUserRepository
7   implements UserRepository {
8   async getUser(id: number) {
9     const rows = await db.query(
10       'SELECT * FROM users WHERE id = ?', [id]);
11     return rows[0] ?? null;
12   }
13 }
```

Usecase

NO SHIP

□□□□□□

```
1 export class GetUserUseCase {
2   constructor(
3     private userRepository: UserRepository
4   ) {}
5
6   async execute(id: number) {
7     const user =
8       await this.userRepository.getUser(id);
9     if (!user) return null;
10
11     // 30 days ago
12     if (user.createdAt < thirtyDaysAgo()) {
13       return null;
14     }
15     return user;
16   }
17 }
```

□□□□□□□□

□□□□□ UserRepository — MySQL □□ Mongo□□□□□□
□□

Presenter NO GRAPH Controller

```
1 // Presenter
2 export class UserPresenter {
3   static toResponse(user: User | null) {
4     if (!user) return { error: 'User not found' };
5     return {
6       id: user.id,
7       name: user.name,
8       created_at: user.createdAt.toISOString(),
9     };
10  }
11 }
```

□□□□□□□□

```
1 // routes/user.ts — 
2 const repo = new MySQLUserRepository();
3 const usecase = new GetUserUseCase(repo);
4 const controller = new UserController(usecase);
5 
6 router.get('/users/:id', controller.getUser);
```

COMPOSITION ROOT

Controller □□□□□□□□□□ Usecase □□□□□□□□ — □□□□□□
□□□□□□

PART 04



MongoDBUserRepository new

```
Presenter toResponse()
```

CacheUserRepository NO GROUPS NO GROUPS MySQLUserRepository NO GROUPS NO GROUPS NO GROUPS NO GROUPS NO GROUPS NO GROUPS NO GROUPS NO GROUPS NO GROUPS NO GROUPS NO GROUPS DBUsecase NO GROUPS NO GROUPS NO GROUPS NO GROUPS

\$ 0000000000 — 00000000000000

CHECKPOINT_01

다음의 코드에서 Cache를 사용하는 클래스는?

A Usecase를 사용하는 클래스

B Controller를 사용하는 클래스

C Repository를 사용하는 CacheUserRepository를 사용하는 DB

FAQ

MVC

MVC Clean Architecture

10 100 1 100

2

Repository

Usecase



Layers separated.

□□□□Spring Boot × DDD□□□□□□□□□□□□□□□□

```
$ curl /users/123
```

```
controller → usecase → repository → presenter
```